

Research Note: Why Low-Level Design Practice is Broken & How to Fix It

Candidate: Priyanshu

Assignment: LLD Practice Platform — Engineering Assignment

Date: September 2026

1. The Core Problem: Why Practicing LLD is Frustrating

When practicing Data Structures & Algorithms (DSA), the feedback loop is crystal clear: your code either passes the 50 hidden test cases or it times out on an edge case.

Low-Level Design (LLD) is completely different. When an engineer sits down to design a **Parking Lot**, an **Elevator System**, or a **Splitwise app**, they usually face three big questions:

- 1. Did I split responsibilities cleanly, or did I accidentally build a massive God-class that handles everything?*
- 2. Is my design actually extensible, or will adding one new requirement break all my classes?*
- 3. If my solution looks completely different from the solution on GitHub, is mine actually wrong, or is it just a valid alternative trade-off?*

Because there are no automated unit tests that can grade "good Object-Oriented Design," learners are left guessing. They usually memorize static solutions from YouTube or GitHub without really understanding *why* certain design choices were made.

2. What I Found When Researching Existing Tools

I spent time analyzing how developers currently prepare for LLD interviews and looked at the four most common options:

1. Static GitHub Repositories (e.g., "awesome-low-level-design")

* **What people do:** Read someone else's Java/C++ code and try to rewrite it from memory.

* **The Problem:** It reinforces the idea that there is only **one golden solution**. If you used the Strategy Pattern instead of the State Pattern, you have no way to know whether your choice was reasonable. It

teaches memorization rather than design thinking.

2. LeetCode "Design" Problems (e.g., Design Underground System, LRU Cache)

4. Product Direction & Approach

To build a focused, practical MVP, I designed the product around three core ideas:

A. A Structured 5-Part Design Canvas

Instead of giving the user a blank code editor (which encourages jumping straight into writing methods without thinking), the canvas asks for the five things an interviewer actually looks for:

7. **Assumptions & Scope:** What is in scope and what is left out?
8. **Class Structure / Model:** What are the domain entities, attributes, and relationships?
9. **Design Patterns & Rationale:** Why was a pattern chosen over simple logic?
10. **Implementation Code:** Clean interfaces, access modifiers, and core methods.
11. **Trade-offs & Concurrency:** How are thread-safety and race conditions handled?

B. A Fixed 6-Dimension Rubric

Every submission is evaluated against 6 concrete design dimensions:

- *Requirement Understanding (15%)*
- *Class Responsibilities & Cohesion / SRP (20%)*
- *Coupling, Interfaces & Encapsulation (20%)*
- *Appropriate Use of Design Patterns (15%)*
- *Extensibility & Trade-offs (15%)*
- *Edge Cases, Concurrency & Testability (15%)*

For each dimension, the feedback must provide:

$\text{Criterion} \rightarrow \text{Score} \rightarrow \text{Evidence (Quote)}$
 $\rightarrow \text{Concern} \rightarrow \text{Suggestion}$

C. Attempt History & Delta Comparison

Every attempt is saved. When a learner submits a second attempt after fixing their code, they can open a **Comparison View** that shows exact score deltas (ΔScore) per dimension, confirming that they actually resolved the previous flaws.

5. Summary

By structuring the submission, evaluating against an evidence-based rubric, and letting learners measure improvement across attempts, this platform turns Low-Level Design from subjective guesswork into a practical, repeatable learning loop.

